1.Requirements

- The system must allow users to register their profile using their student number.
- The system must allow users to add and post the textbook with full details like price, ISBN, textbook name and module code.
- The system must allow users to search for the name of the book using keywords such as module code, or title
- The system must allow users to book an appointment with the seller.
- The system must not proceed if they are invalid inputs or the is blank space.

2. System Architecture Rationale

The MVVM pattern is composed of three main parts, each with distinct responsibilities:

- Model: this layer represents the application's data, business logic, and data access operations.
- View: the View is the user interface (UI) layer, responsible for the structure, layout, and appearance of what the user sees on screen.
- ViewModel: this acts as the link between the View and the Model. It contains presentation logic, exposes data streams and commands to the View, and notifies the View of any state changes.

How MVVM Supports State Management

The MVVM supports the state management by creating a strict separation between the user interface (View) and the underlying data/business logic (Model), using the ViewModel as a reactive intermediary. Separation of Concerns of MVVM, View (UI) acts passively, merely rendering the state provided by the ViewModel and sending user intents (clicks, inputs) back to it. ViewModel (Logic) will manage all presentation logic, such as transforming, sorting, or filtering data from the Model before exposing it to View. Model (Data) will handle data fetching and business logic. To reduce UI bugs.

How MVVM Enables Testing Readiness

MVVM architecture significantly enhances testing readiness by providing a clear separation of concerns, which is crucial for effective testing. The Model, View, and ViewModel work together to ensure that the UI is clean and focu sed on presentation, without any business logic. Making it possible for the ViewModel to be unit-tested independently, reducing the dependence on Android framework libraries. The testing will mainly focus on appointment scheduling and search filtering logic.

Why No Database is Required for MVP

MVP does not need database because it uses memory data structures. You can use hardcoded sample data to simulate functionality. Speed of development like avoiding database setup reduces complexity, allowing faster iteration and quicker feedback from users. Does not require persistent storage.

3. Main App Components

- **Registration Screen (UI Only)**

The registration screen is the front-end interface where new users provide their details to create an account. It focuses on the layout. The core elements will be the input field take in the student's name and student number, the button to register and visual design a clean layout. To serve the purpose of a user-friendly entry point for account creation. As they won't be any backend authentication server for UI.

- **Book Listing Interface**

The purpose is to display available books in a structured form. Each book will include each field such as title, author, module code and ISBN. To maintain accuracy and usability, the system will incorporate form validation so that only properly completed entries are accepted.

- **Search Functionality**

The purpose of search functionality is to allow buyers to quickly find a certain textbook. The components will be the search bar for entering the keyword like ISBN or title to find the book. Real-time filters. The search login will be implemented inside the ViewModel using in-memory data that is often backed by indexing to ensure fast retrieval.

- **Inquiry Mechanism**

The purpose of inquiry mechanism is to allow both buyers and sellers to interact. The components of Inquiry Mechanism will be the contact form, contact button to call the seller, confirmation message for feedback without the use of backend messaging system.

- **Navigation Flow**

The purpose of navigation flow is to guide users to navigate smoothly through the app. This navigation flow will be using a single activity architecture from one section to another. The navigation flow for the app will be Registration → Home → Listing Detail → Inquiry / Appointment → Confirmation.

- **In-Memory Data Handling**

The purpose of In-Memory Data Handling is to handle the temporary data during the sessions. This implementation is responsible for storing and managing textbook entities, supporting basic operations such as adding and removing items. By simulating backend storage entirely in memory, this design avoids external dependencies.

- **Error Handling**

The purpose of error handling is to make sure that the system is stable, and it gives a clear error message when issues occur. The system will handle empty form field using feedback mechanisms like Inline form errors, for example if you don't enter anything it will give you an error such as no search results found and will allow users to refresh the page.

- **UI Responsiveness**

The purpose for UI responsiveness is to automatically adjust the layout and content to fit screens from mobile phones to desktops for the content to fit perfectly, reducing the need for zooming or scrolling, and improving usability. Ensuring smooth performance on entry-level devices